

C Foundation -- Updated Plan

Phase 1 — Foundation

- Introduction to C
- Structure of a C Program
- Keywords and Identifiers
- Variables
- Data Types
- Constants
- Header Files
- Input and Output (printf, scanf)
- Compilation process

Phase 2 — Logic Building & Flow Control

Operators: Arithmetic, Relational, Logical, Assignment, Increment/Decrement

Conditional Statements: if, if-else, nested if, switch

Loops: for, while, do-while

Jump Statements: break, continue, goto

Phase 3 — Functions

Function declaration, definition, call, Call by Value and Call by Reference

Phase 4 — Data Collections

Arrays: 1D, 2D | Strings

Phase 5 — Hard Problems and Doubt Clearing

Questions

Phase 1 — Foundation

Introduction to C / Hello World / Program Structure

1. Write a program that prints your name and city on two separate lines.
2. Write a program that prints a simple box border using `*` characters.
3. Add a single-line and a multi-line comment to a Hello World program explaining what each line does.

Keywords, Identifiers, Variables

1. List 5 valid and 5 invalid variable names and explain why each is valid/invalid.
2. Declare variables to store a student's name, age, grade, and fees — choose appropriate types for each.
3. **Swap the values of two integer variables without using a third variable.**
4. **Declare a variable, print it without initializing it, and observe/explain the output.**

Data Types

1. Declare one variable of each basic type (`int` , `float` , `double` , `char`) and print them.
2. Print the size of each data type using `sizeof()` .
3. Store the value 3.14159 in both a `float` and a `double` and compare the printed precision.
4. Find the maximum value an `int` can hold on your system and print it.

Constants

1. Define a constant for PI and use it to calculate the area of a circle.
2. Use `#define` to set the number of days in a week, then print a multiplication table for it.
3. Rewrite a program that uses magic numbers (like 360 , 7 , 24) by replacing them with named constants.

Input and Output

1. Read your name and age from the user and print a greeting.
 2. Read two integers and print their sum, difference, product, and quotient.
 3. Read a float and print it with 2, 4, and 6 decimal places using format specifiers.
 4. Read a character and print its ASCII value.
 5. Read three subject marks and print them in a neatly formatted table with labels.
-

Phase 2 — Logic Building & Flow Control

Value Operators (Arithmetic, Relational, Assignment, Increment/Decrement)

1. Read two numbers and print the result of all arithmetic operations on them.
2. Read an integer and print whether it is even or odd using the `%` operator.
3. Predict the output of `x = 5; y = x++ + ++x;` before running it, then verify.
4. Read a 3-digit number and extract each digit using `/` and `%` .
5. Calculate simple interest given principal, rate, and time using a single formula.
6. Check whether a number is **automorphic** — a number whose square ends with the number itself (e.g. $5^2 = 25$, $6^2 = 36$). Use `%` to extract the last digits.

Logical Operators

1. Read an integer and check if it is between 1 and 100 (inclusive) using `&&` .
2. Read a character and check if it is a vowel using `||` .
3. Read a year and check if it is a leap year (covers both `&&` and `||` in one problem).
4. Read three numbers and check if at least two of them are equal.

Conditional Statements

1. Read a number and print whether it is positive, negative, or zero.
2. Read a student's mark and print the grade (A/B/C/D/F) using `if-else if` .
3. Read three numbers and print the largest using nested `if` .
4. Read a number 1-7 and print the corresponding day of the week using `switch` .
5. Build a simple calculator (add/subtract/multiply/divide) using `switch` with a menu.

Loops

1. Print numbers 1 to 100 using a `for` loop.
2. Print the multiplication table for any number entered by the user.
3. Read numbers from the user until they enter 0, then print the sum.
4. Check whether a number is **prime** — read a number and determine if it has any divisors other than 1 and itself.

- Print all prime numbers between 1 and 50.
- Check whether a number is a **perfect number** — a number equal to the sum of its proper divisors (e.g. $6 = 1+2+3$). Print all perfect numbers up to 1000.
- Check whether a number is an **Armstrong number** — a number equal to the sum of its digits each raised to the power of the number of digits (e.g. $153 = 1^3+5^3+3^3$).
- Print all Armstrong numbers up to 1000.
- Print **Floyd's Triangle** — a right triangle of sequential numbers starting from 1:

```
1  1
2  2 3
3  4 5 6
4  7 8 9 10
```

- Print the following **Equilateral Triangle** pattern for a given number of rows N (hint: combine leading spaces with stars):

```
1      *
2     * *
3    * * *
4   * * * *
5  * * * * *
```

- Print the following right-angle star pattern using nested loops:

```
1  *
2  * *
3  * * *
4  * * * *
```

Break and Continue

- Print numbers 1–20 but skip multiples of 3 using `continue`.
- Read numbers in a loop and stop as soon as a negative number is entered using `break`.
- Search for a number in the range 1–50 and break when you find the first one divisible by both 6 and 8.
- Print all numbers 1–100, skip even numbers, and stop at the first number greater than 50 that is divisible by 7.

Phase 3 — Functions

- Write a function `isEven(int n)` that returns 1 if even, 0 if odd. Call it in a loop to print all even numbers up to 50.
- Write a function `power(int base, int exp)` that returns base raised to exp without using `pow()`.
- Write a function `isPrime(int n)` and use it to print all primes up to N.
- Write a recursive function `factorial(int n)` and compare its output with an iterative version side by side.
- Write a function `calculator(float a, float b, char op)` that takes two numbers and an operator (+, -, *, /) and returns the result.
- Write a function `gcd(int a, int b)` that returns the GCD of two numbers using the Euclidean algorithm. Test it with at least 3 pairs of numbers.
- Write a function `lcm(int a, int b)` that internally calls your `gcd()` function using the relation $LCM(a,b) = (a*b) / GCD(a,b)$.

8. Write a function `isStrong(int n)` that checks whether a number is a **Strong number** — a number where the sum of the factorials of its digits equals the number itself (e.g. $145 = 1! + 4! + 5!$). Reuse your `factorial()` from problem 4 inside it.
-

Phase 4 — Data Collections

1D Arrays

1. Read 10 integers into an array and print them in reverse.
2. Find the maximum, minimum, and average of elements in an array.
3. Read an array and count how many elements are greater than the average.
4. Write a function that takes an array and its size and returns the second largest element.
5. Implement linear search — read a target from the user and print its index or "not found."

2D Arrays

1. Read a 3×3 matrix and print it in matrix format.
2. Find the sum of each row and each column of a matrix.
3. Add two 3×3 matrices and print the result.
4. Find the transpose of a matrix.
5. Check whether a given 3×3 matrix is symmetric.
6. Generate and print **Pascal's Triangle** up to N rows — each cell is the sum of the two cells directly above it (`triangle[i][j] = triangle[i-1][j-1] + triangle[i-1][j]`).

Strings

1. Read a string and print its length without using `strlen()`.
 2. Read a string and print it reversed.
 3. Check whether a given string is a palindrome.
 4. Read a sentence and count the number of vowels, consonants, and spaces.
 5. Read two strings and concatenate them without using `strcat()`.
-

Phase 5 — Problem Practice

Practice Frequently Asked Questions

- Sum and product of all elements in a matrix
- Fibonacci series (both iterative and recursive)
- All Armstrong numbers up to 1000
- All Strong numbers up to 10000
- All perfect numbers up to 1000
- GCD and LCM of a list of numbers entered by the user
- Pascal's Triangle for N rows with proper formatting
- All prime numbers in a range using `isPrime()` function
- Palindrome check for both numbers and strings
- Sort an array using bubble sort